

АПиТCUDA Л.Р. №3

Стракович Андрей Иванович

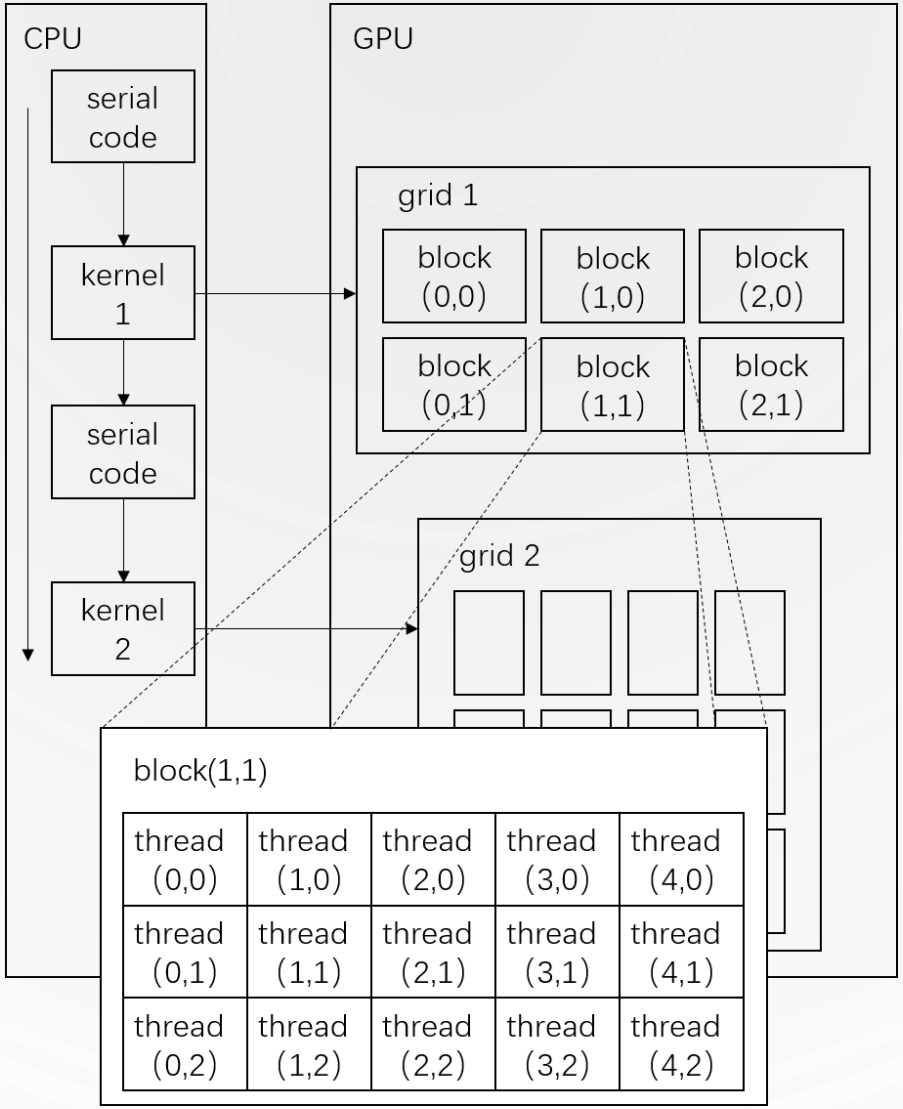
БГУИР, кафедра ЭВМ

а. 508-5

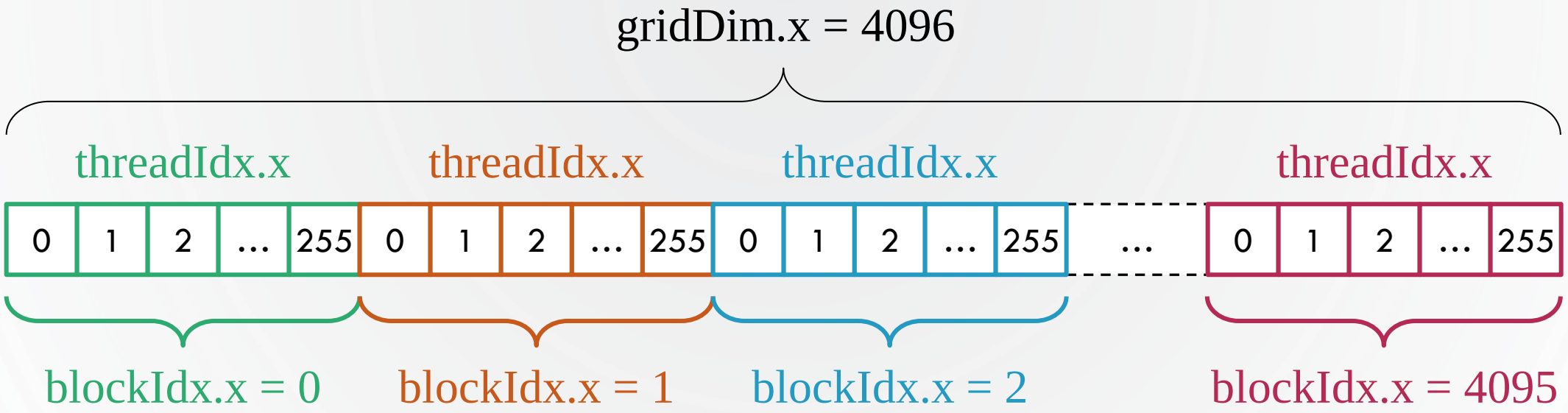
2026

ВВЕДЕНИЕ В ТЕХНОЛОГИЮ CUDA

CUDA

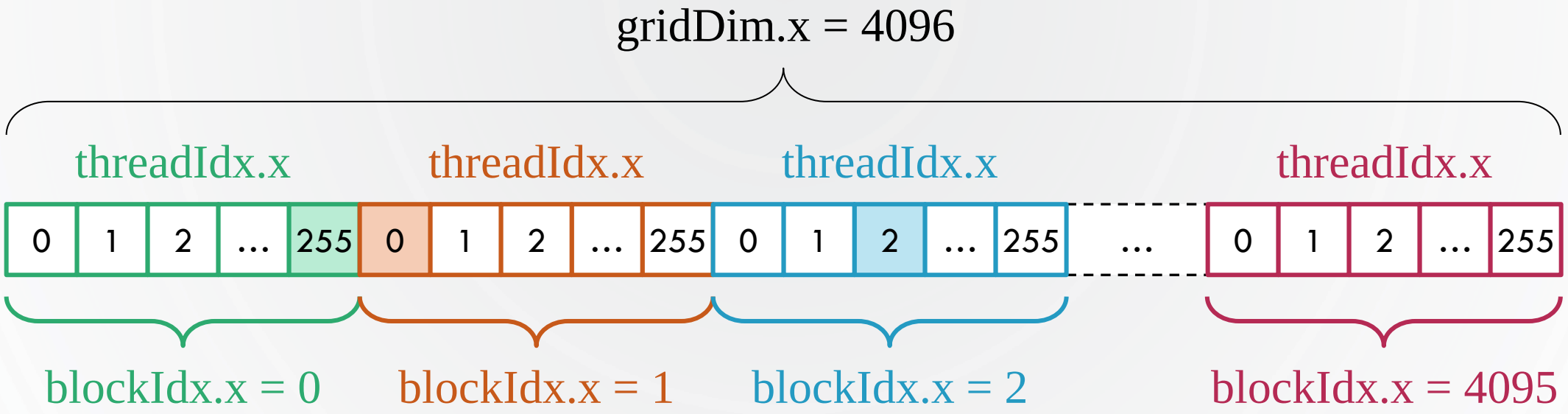

CPU
serial code
kernel 1
serial code
kernel 2
GPU
grid 1
block (0,0)
block (1,0)
block (2,0)
block (0,1)
block (1,1)
block (2,1)
grid 2
block(1,1)
thread (0,0)
thread (1,0)
thread (2,0)
thread (3,0)
thread (4,0)
thread (0,1)
thread (1,1)
thread (2,1)
thread (3,1)
thread (4,1)
thread (0,2)
thread (1,2)
thread (2,2)
thread (3,2)
thread (4,2)

CUDA



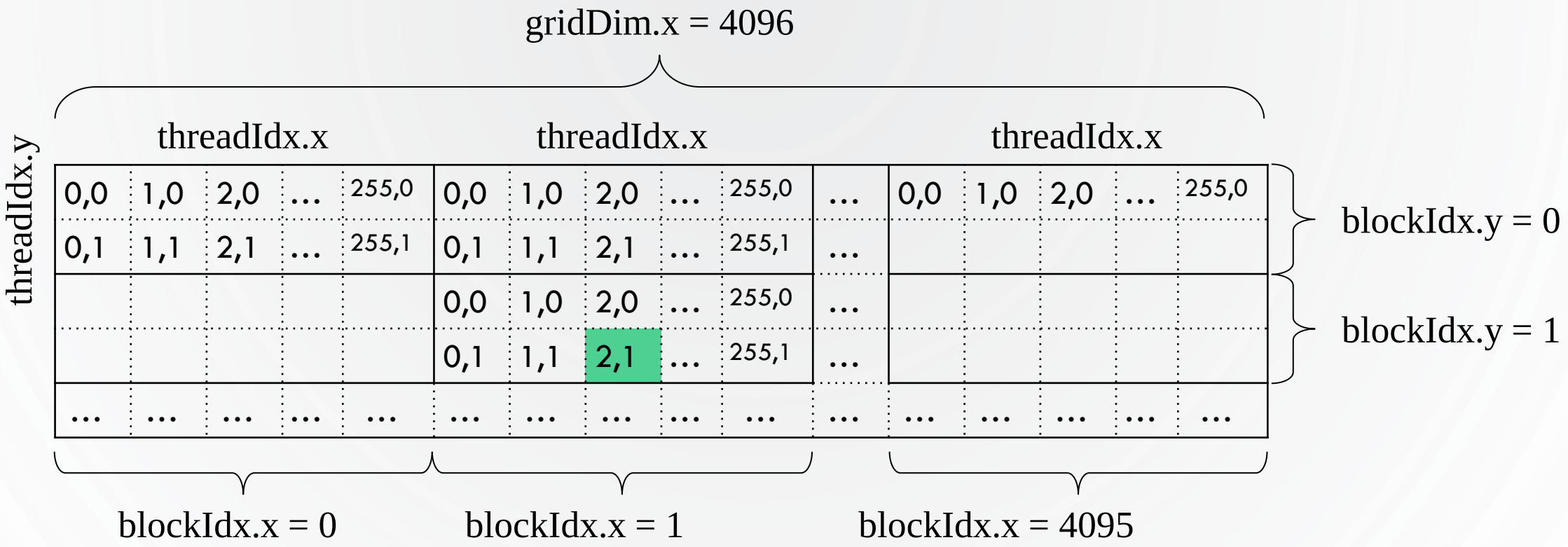
index = blockIdx.x * blockDim.x + threadIdx.x

CUDA



index	=	blockIdx.x	*	blockDim.x	+	threadIdx.x		
index	=	0	*	256	+	255	=	255
index	=	1	*	256	+	0	=	256
index	=	2	*	256	+	2	=	514

CUDA

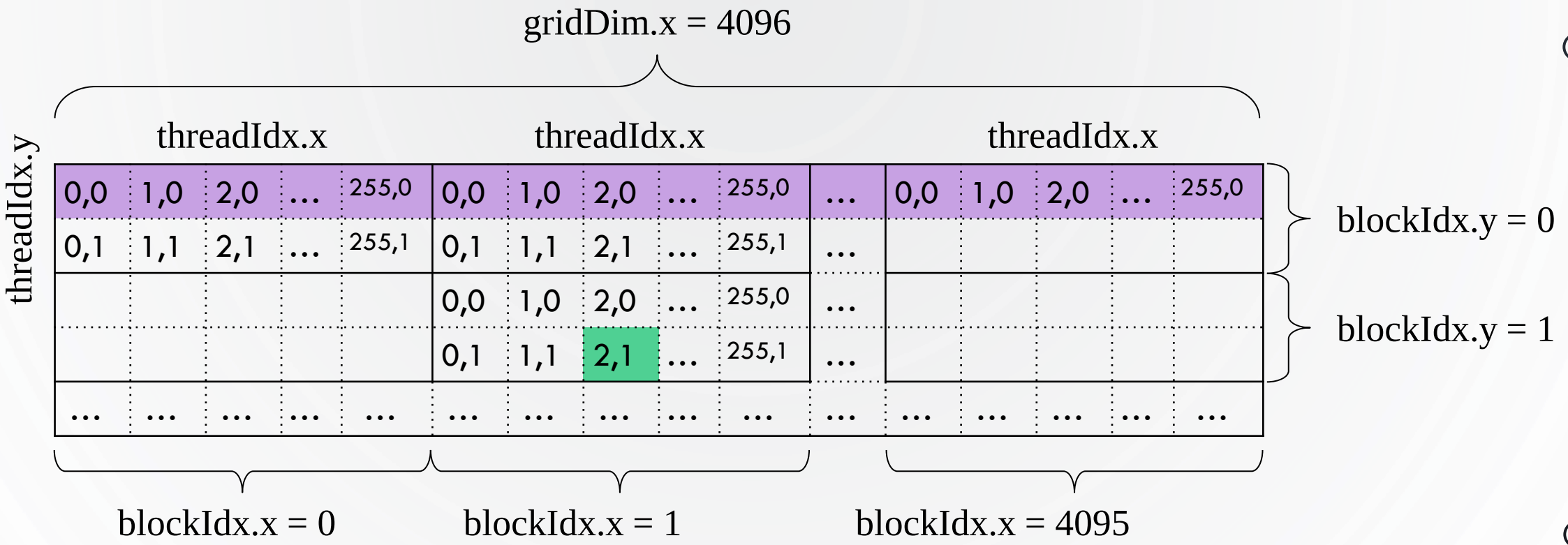


indexX = blockIdx.x * blockDim.x + threadIdx.x = 1 * 256 + 2 = 258

indexY = blockIdx.y * blockDim.y + threadIdx.y = 1 * 2 + 1 = 3

offset = indexY * gridDim.x * blockDim.x + indexX = 3 * 4096 * 256 + 258 = 3145986

CUDA

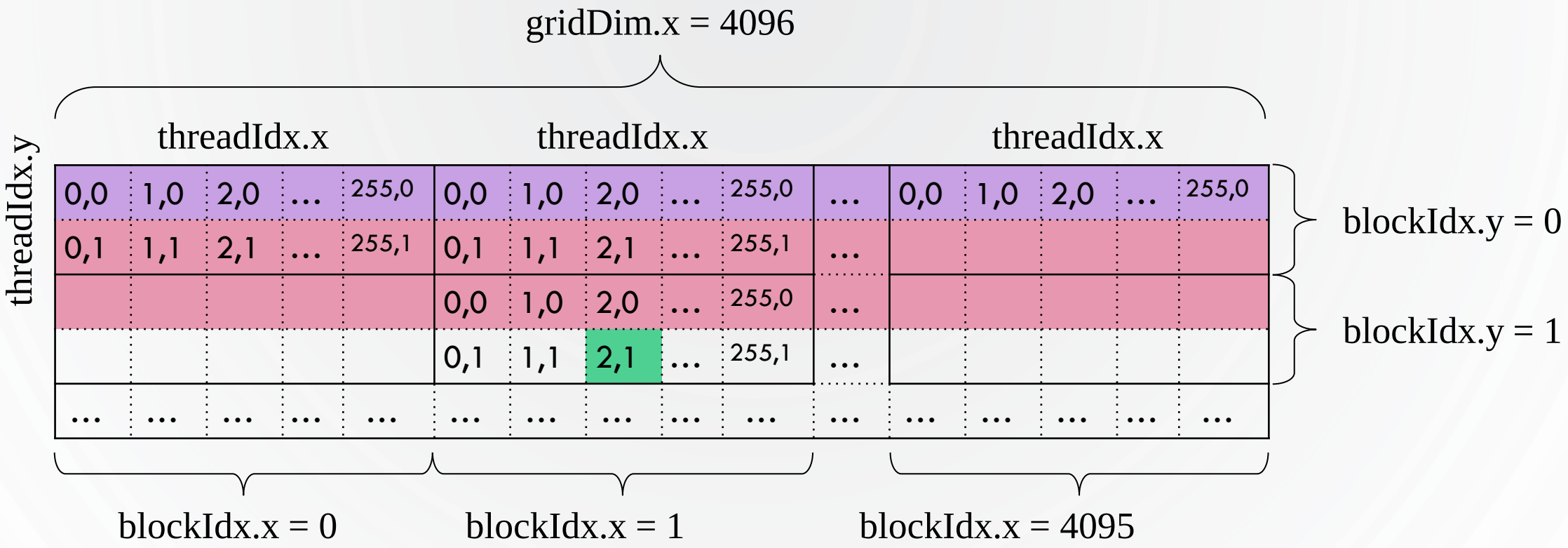


indexX = blockIdx.x * blockDim.x + threadIdx.x = 1 * 256 + 2 = 258

indexY = blockIdx.y * blockDim.y + threadIdx.y = 1 * 2 + 1 = 3

offset = indexY * gridDim.x * blockDim.x + indexX = 3 * 4096 * 256 + 258 = 3145986

CUDA

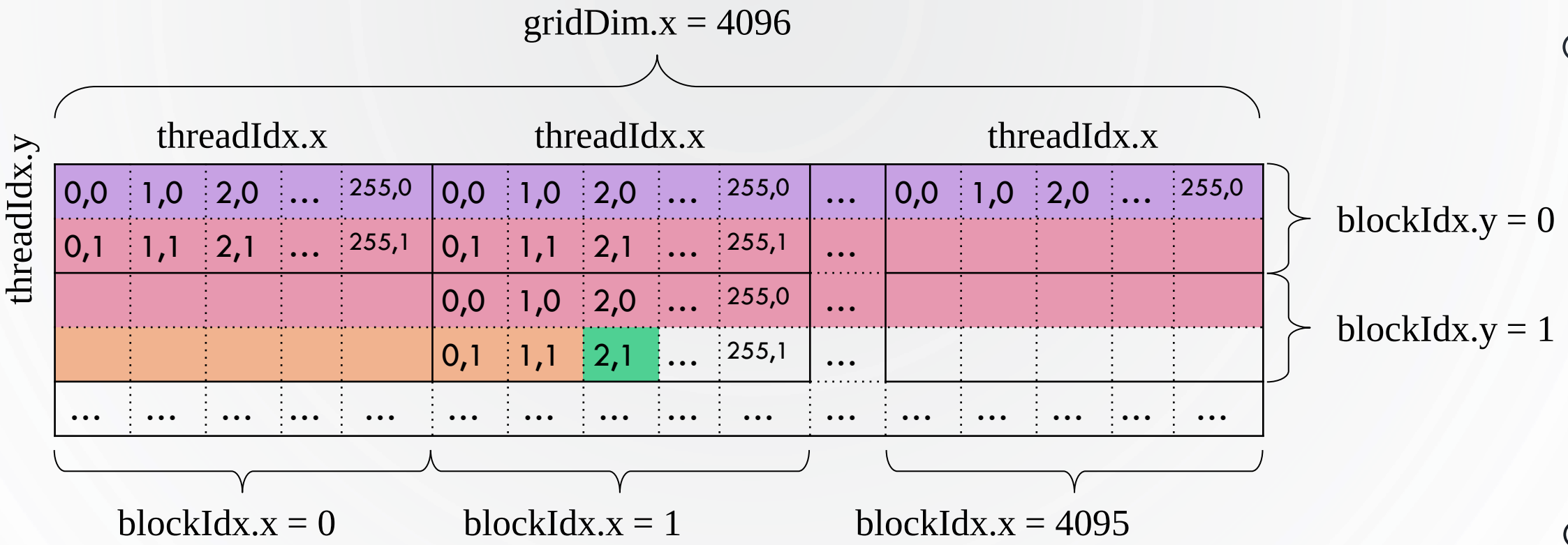


indexX = blockIdx.x * blockDim.x + threadIdx.x = 1 * 256 + 2 = 258

indexY = blockIdx.y * blockDim.y + threadIdx.y = 1 * 2 + 1 = 3

offset = indexY * gridDim.x * blockDim.x + indexX = 3 * 4096 * 256 + 258 = 3145986

CUDA



indexX = blockIdx.x * blockDim.x + threadIdx.x = 1 * 256 + 2 = 258

indexY = blockIdx.y * blockDim.y + threadIdx.y = 1 * 2 + 1 = 3

offset = indexY * gridDim.x * blockDim.x + indexX = 3 * 4096 * 256 + 258 = 3145986

ТРЕБОВАНИЯ

- Реализация CPU и GPU
- Сравнение времени работы в **едином формате**
  - формат: микро-, миллисекунды и т.п.
  - измерение времени на GPU **через события** в CUDA
- Сравнение результатов работы:
  - полное поэлементное сравнения массива
  - вывод фрагмента на экран

РЕАЛИЗАЦИЯ GPU

- Для сдачи л.р. достаточно простой реализации, но можно сделать дополнительную оптимизацию
- Каждая оптимизация – отдельное ядро, основанное на **предыдущей** реализации
  - корректное обращение в память
  - работа со сверхбольшими объемами данных (критерий - нехватка объема памяти GPU раза в 2-3 хотя-бы)
    - можно сдавать на КП в аудитории
  - применение CUDA Stream
    - должно быть ускорение в сравнении с другими версиями GPU
    - занять почти всю память видеокарты
  - можно задать любой размер входной матрицы

ЗАДАНИЕ

- По варианту сделать

- Если результат можно представить в виде изображения:
  - вывести на экран (опционально)
  - сохранить в виде изображения (опционально)